

15/03/2023

Version 1



TDD

JÉRÉMY PERROUAULT



TESTS

TESTS UNITAIRES

TESTS UNITAIRES

- Pour nous aider dans ces tâches
 - Tests unitaires
 - Tests d'intégration
 - Tests de bout en bout (end-to-end)
- Un test unitaire doit être
 - Simple / Lisible
 - Isolé / Indépendant
 - Rapide à s'exécuter

TESTS UNITAIRES

- La qualité d'un test unitaire
 - C'est la capacité à déterminer le nombre de tests unitaires pour une méthode
- Exemple : méthode avec une condition « `if – else` »
 - Deux états
 - Au moins deux tests unitaires (un pour chaque état)

TESTS UNITAIRES

- Pour les tests unitaires en **JAVA**
 - Utilisation de **JUnit**
 - Dépendance **Maven** junit-jupiter-api

EXERCICE

- Rédaction d'un code et de son ou ses tests unitaires
 - En respectant des règles énoncées
- Ajouter une classe `Calculatrice`
 - Fonctionnalités `additionner` et `soustraire`

A decorative graphic on the left side of the slide consisting of two parallel, wavy vertical lines. The inner line is a light blue color, and the outer line is white. They start from the top left and extend towards the bottom left, creating a stylized, organic shape.

TDD

TEST DRIVEN DEVELOPMENT

TDD

- L'approche classique du test
 - Développer la fonctionnalité
 - Tester la fonctionnalité
 - Plus le bug est détecté tard, plus il remet en cause d'éléments, et sa correction peut être coûteuse
 - Le code étant déjà écrit, le risque est d'écrire un test qui valide le code (un test faux)
- L'approche **TDD**
 - Développer le test
 - Développer la fonctionnalité
 - Tant que le test n'est pas validé, la fonctionnalité n'est pas valide
 - Cycle *RED* → *GREEN* → *REFACTOR*

TDD

- L'exécution d'un test doit, en premier lieu, donner un résultat négatif
 - La rédaction du code qui s'en suit doit valider progressivement le ou les tests
 - En respectant toujours des principes vus précédemment
 - Un refactor peut suivre
 - Principe « Red-Green-Refactor »

TDD

- Deux approches
 - Tout d'un bloc : un ou plusieurs tests pour l'ensemble des cas
 - Généralement pour les algorithmes facilement compréhensibles / dont on maîtrise déjà la finalité
 - Quelques cas simples, et les cas les plus complexes
 - Rédaction du code jusqu'à ce que tout passe au vert
 - Petits cycles itératifs
 - Rédiger le test d'un cas simple
 - Rédiger le minimum de code possible pour faire passer le test
 - Enrichir le test d'un deuxième cas simple
 - Compléter la fonctionnalité pour valider les deux tests
 - Refactoriser le code
 - Enrichir le test d'un troisième cas, plus complexe
 - Compléter la fonctionnalité pour valider les trois tests
 - Refactoriser le code
 - etc. jusqu'à couvrir tous les cas

TDD

- Peu importe l'approche, chaque test reste
 - Simple
 - Isolé
 - Indépendant
 - Rapide
- Et on a toujours plusieurs tests pour valider une fonctionnalité

TDD

- Avantages techniques
 - Le temps de débogage est considérable réduit
 - La détection de bugs est plus immédiate
 - Les tests de non-régression sont déjà prêts
 - L'architecture est testable et de bonne qualité
 - On réduit le risque de valider un code faux par un test faux
 - Les tests peuvent ainsi être utilisés comme documentation
 - Evite également les étourderies liées au copier-coller de code (réguliers ...)
 - Evite d'oublier de déclarer la méthode de test comme étant une méthode de test (**@Test**)

TDD

- Le **TDD** est bien du test, avec une philosophie différente
 - « Tester c'est construire » plutôt que « Tester c'est douter »

EXERCICE

- **JAVA (JUNIT)**
 - Créer une classe `Calc`
 - Ajouter une méthode `add` qui doit additionner 2 `String` reçus en paramètres et retourne un entier

EXERCICE

- **JAVA (JUNIT)**

- Reprendre la classe `Calc`

- Ajouter une méthode `add` qui attend un seul `String` et retourne un entier
 - Si chaîne vide ou null, retourner 0
 - "2" retourne 2
 - "12,5,19" retourne 36
 - Un chiffre négatif jette une exception `NegativeNotAllowedException`
 - Prendre en compte les points-virgules "12;5;19"
 - Prendre en compte les sauts de lignes, l'exemple qui suit doit retourner 36 (saut de ligne = `\n`)

12

5,19

EXERCICE

- **JAVA (JUNIT)**
 - Fizz Buzz FizzBuzz
 - Pour des entiers de 1 à 15
 - Les multiples de 3 sont remplacés par « Fizz »
 - Les multiples de 5 sont remplacés par « Buzz »
 - Les multiples de 15 sont remplacés par « FizzBuzz »
 - 12Fizz4BuzzFizz78FizzBuzz.....